



Decision Theory II

Reflective Oracles, Open-Source Games, and Commitment Races

Daniel C



Where we are: agents that model each other

Two foundational obstacles to mutual modelling

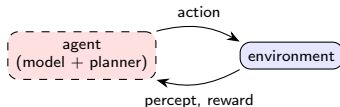
- Deck I ended at **fair computational worlds**: agent and environment are both programs that may inspect, simulate, or predict each other. Embedded agents must model other agents – including agents like themselves.
- This raises two questions, and this deck takes both:
 - (1) **Can a Bayesian agent even represent other agents like itself?** Self-modelling failures of AIXI, the *grain of truth* problem, and *reflective oracles*.
 - (2) **When agents can read each other's code, how do they cooperate without conflict?** Open-source game theory, *commitment races*, and *safe Pareto improvements*.
- Punchline preview: reflective oracles solve mutual modelling *semantically* (a shared oracle); open-source games solve it *syntactically* (read the code). Both then face the strategic problem of *commitment*.



The cybernetic model and large agents

Learn a model, then plan with it

- Cybernetic model: an agent *learns a model* of its environment and *plans* with it to achieve a goal.
- Two implicit assumptions:
 - (1) the agent is “**larger than**” the environment – its mind can hold a full copy of the world;
 - (2) the agent is “**outside**” the environment.
- Note 1 $\Rightarrow$ 2, so model-based RL almost always assumes (2). Can we still reason when the agent is *smaller* than its environment?



clean boundary: the agent sits outside the world

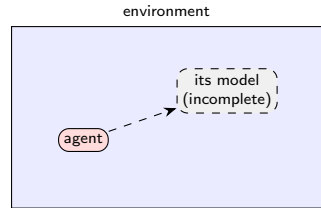
(Section follows C. Wyeth, *Embedded Agency*; diagrams recreated, after the LessWrong Embedded Agency sequence.)



The actual situation: embedded agency

The agent is a part of the world

- An agent is *just a special part of* the environment.
- It may be **modified, destroyed, or copied** (by itself or others) between interaction steps.
- Other **equally powerful agents** may exist, *mutually reasoning about each other*.
- Safety relevance of getting this right: reward hacking, the shutdown problem, user identification.
- Can we reason about an agent *smaller* than, and contained in, its environment?



it cannot fit the whole world (including itself)

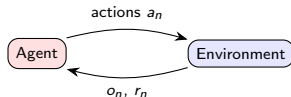
inside



History-based reinforcement learning

The general framework

- Under the cybernetic assumption, a very general class of tasks is modelled as **history-based RL**.
- The agent emits actions a_n and receives percepts $e_n = (o_n, r_n)$: observations o_n and rewards r_n .
- The *interaction history* is the interleaved sequence $a_1 o_1 r_1 a_2 o_2 r_2 \dots$



$\dots a_1 o_1 r_1 a_2 o_2 r_2 a_3 o_3 r_3 \dots$
→ time



The optimal history-based agent for computable environments [Hut00]

- AIXI **plans optimally against a Bayesian mixture of all programs**: a Solomonoff prior over environment-programs q , weighted by simplicity $2^{-\ell(q)}$.
- The simplest formulation fits on one line (fixed horizon m , iterative value function), writing $e_i = o_i r_i$:

$$a_k := \arg \max_{a_k} \sum_{o_k r_k} \cdots \max_{a_m} \sum_{o_m r_m} [r_k + \cdots + r_m] \sum_{q: U(q, a_{1:m})=e_{1:m}} 2^{-\ell(q)}.$$

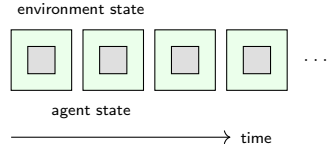
- Read right-to-left: weight every environment-program q consistent with the chosen actions by $2^{-\ell(q)}$, average the reward, then *expectimax* over actions and percepts. The gold standard of an idealised *dualistic* agent – but is it *embedded-ready*?



Joint history prediction: the idea

Predict the whole tape, actions included

- Go *beyond* the cybernetic framework by **jointly predicting the entire history** – actions and percepts alike.
- Formalisation: $a_1 e_1 a_2 e_2 \dots = h \sim \rho$ for a distribution ρ over histories.
- The cybernetic assumption is the *special case* $\rho = \mu^\pi$: the history factorises into an environment μ (percepts given actions) and a policy π (actions given history).





Joint history prediction: Solomonoff's mixture

From pure prediction to a joint belief

- For pure prediction, Ray Solomonoff's universal distribution is a Bayesian mixture over all probabilistic programs:

$$M(x) = \sum_{p: U(p)=x*} 2^{-\ell(p)} \quad (\text{programs whose output starts with } x).$$

- Marcus Hutter extended this to decision problems. The joint AIXI belief is built from M as a product of one-step conditionals:

$$\xi^U(e_{1:t} \parallel a_{1:t}) = \prod_{i=1}^t M(e_i \mid h_{<i} a_i).$$

- Why not just use this to predict the joint distribution – actions and all?*



Joint history prediction: why it breaks for RL

Joint prediction does not transfer cleanly

- Joint prediction does **not** immediately transfer to RL; learning may **not converge** even when the cybernetic assumption holds.
- One can choose **uncomputable odd bits** that destroy Solomonoff prediction on the *computable* even bits, e.g.

11 00 00 11 11 11 11 ... (odd bits uncomputable, even bits computable).

- The same trick: certain **uncomputable actions** can break Solomonoff *percept* prediction.
- Surprisingly, **normalising M** patches this particular problem.



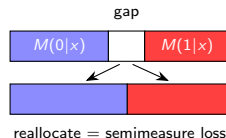
Semimeasure loss

The missing probability mass

- M is only a **semimeasure**: probability can *leak* (as if the sequence may "stop"),

$$\nu(x) \geq \sum_{a \in \Sigma} \nu(xa) \quad (\text{a measure has equality}).$$

- There are several ways to *complete* / *normalise* a semimeasure: reallocate the leftover gap among the continuations.
- The gap reallocated this way is the **semimeasure loss**.





Prediction of selected bits

The semimeasure loss can be large infinitely often

- The semimeasure loss can be made *large infinitely often* [Lattimore–Hutter–Gavane].
- **Theorem 7 (adversarial non-convergence of ξ^U)** [Wyeth–Hutter]: there exists $e = a \in \mathbb{B}^\infty$ such that

$$\xi^U(e_{1:t} \parallel a_{1:t}) = \xi^U(a_{1:t} \parallel a_{1:t}) \longrightarrow 0 \quad \text{as } t \rightarrow \infty.$$

- Reading: there are histories where, even though the reward has *always* matched the (binary) value of the action, AIXI **infinitely often doubts** this will continue.
- **Caution:** we do not know whether these (action) histories are *on-policy* for AIXI. And Theorem 7 **fails** when M is normalised.



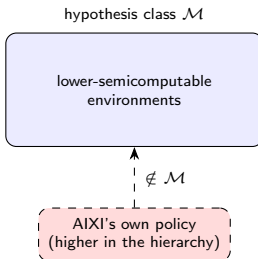
AIXI is much “larger” than its environment

The agent is not in its own hypothesis class

- AIXI believes the environment is **lower semicomputable** (Σ_1^0).
- But AIXI's own *policy* is **not** lower semicomputable – it sits *higher* in the arithmetical hierarchy:

$$\dots \Delta_n^0 \subset \Sigma_n^0, \Pi_n^0 \subset \Delta_{n+1}^0 \subset \Sigma_{n+1}^0, \Pi_{n+1}^0 \dots$$

- Hence “**AIXI does not believe the universe contains AIXI agents.**”





Modeling multi-player games

Mutual modelling needs same-size agents

- Agents which are modelling each other must be of the “**same size**”.
- Therefore AIXI **cannot learn** that the environment contains other AIXI instances.
- Note: multi-player games do *not* necessarily involve other embedded-agency problems (“being inside the environment”) – this obstacle is specifically about *mutual modelling*.



*each must fit a model of the other – as powerful as
itself*



The grain of truth problem

Correctly-specified subjective models

- In a multiplayer game we want each Bayesian player to have a *correctly-specified* model of its subjective environment: never “**infinitely surprised**” by an event that has positive probability under the true history distribution.
- Given a class of games $\mathcal{G}$, a class of strategies $\mathcal{P}$, and subjective beliefs ξ_i for player i , the **grain of truth property** requires:
 - each optimal policy $\pi_{\xi_i}^* \in \mathcal{P}$ (the best response stays in the class), and
 - each belief $\xi_i \gg \sigma^\pi$ for all $\sigma \in \mathcal{G}$, $\pi \in \mathcal{P}^n$,

where $\gg$ denotes **absolute continuity** (the truth is absolutely continuous w.r.t. the belief; by Blackwell–Dubins, posteriors then *merge*).



Solution with reflective oracles (I): the construction

Precise access to others, without hanging

- This appears to **fail for AIXI players**. Naively, build games/policies by

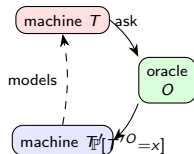
$$\lambda_T(x_t \mid x_{<t}) = \mathbb{P}[T(x_{<t}) = x_t],$$

but **policies hang** when they call each other (infinite regress).

- Instead, give each machine T a **reflective oracle** O giving *arbitrarily precise access* to the RHS for all other machines T' that also use O :

$$\lambda_T^O(x_t \mid x_{<t}) = \mathbb{P}[T^O(x_{<t}) = x_t].$$

- O reflects the true probability and may **randomise** exactly at the query threshold, so nothing hangs and no liar paradox arises (such O exists – Fallenstein–Taylor–Christiano). These augmented machines build a **reflective AIXI** with the grain of truth property.



randomise at the threshold $\Rightarrow$ no hang



Solution with reflective oracles (II): results

What reflective oracles buy

- **Game-theoretic results:**
 - in the “dualistic” multiplayer setup: convergence to **Nash equilibrium** in stage games, and generally convergence to optimality for **Thompson sampling** [Leike–Taylor–Fallenstein; Wyeth et al.];
 - if we allow *correlations* between policies (joint prediction): provable “**acausal**” **coordination** in the Prisoner’s Dilemma [Meulemans et al.].
- The **semimeasure loss is eliminated** and prediction of selected bits *succeeds*.



Solution with reflective oracles (III): self-modelling

Using the self-model – and its limits

- An agent can **exploit its self-model to avoid planning**; in principle this lets it *reason about self-modifications*.
- However, it is **difficult to prove optimality** (or any other interesting property) of this model. With the oracle-based joint belief $a_1 e_1 a_2 e_2 \dots \sim \xi^O$:

$$\pi_S(h_{<t}) \in \arg \max_{a_t} \xi^O \left(\sum_{i=t}^{\infty} \gamma_i r_i \mid h_{<t} a_t \right).$$

- **Bridge to the rest of the deck:** reflective oracles solve mutual modelling *semantically* (a shared oracle); open-source game theory does it *syntactically* – the agents read each other's source code.



A 60-second refresher on game theory

Nash, and the Prisoner's Dilemma

- A (normal-form) game: players, action sets, and payoffs $u_i(a_1, \dots, a_n)$ depending on the *joint* action.
- **Nash equilibrium:** a profile where no player gains by deviating *alone*.
- **Prisoner's Dilemma:** (C, C) is better for both than (D, D) , yet D *dominates* C for each player. The unique Nash is (D, D) : rational players defect, and both end up worse.
- **Root cause:** your payoff depends on the *joint* action, but you must choose *without knowing or affecting* the other's choice. You cannot *condition* on what they do.

		Player 2	
		C	D
P1	C	3, 3	0, 5
	D	5, 0	1, 1

(D, D) is the only Nash, though (C, C)
beats it for both.



Löbian cooperation: FairBot

When players can read each other's code

- **Open-source (program) game** (Tennenholtz): each player submits a *program* that takes the opponent's *source code* as input and outputs an action. Now you *can* condition on the opponent.
- **FairBot**: “cooperate iff I can *prove* my opponent cooperates with me.”
- **FairBot vs. FairBot**: by **Löb's theorem**, each proves mutual cooperation and so cooperates. (Here Löb is a *feature*: $\Box(\Box C \rightarrow C) \rightarrow \Box C$, the self-referential proof search succeeds.) Two mutually suspicious agents cooperate, just by reading each other.
- **Unexploitable**: FairBot cooperates with CooperateBot, but *defects* against DefectBot (it cannot prove a defector cooperates). It is never the sucker.
- Caveat: proof-based FairBot is *brittle* – it depends on syntactic provability, can fail to prove cooperation with a differently-written but behaviourally-cooperative bot, and proof search may not terminate.



Robust cooperation by simulation: ε -grounded bots

Replacing proofs with simulation (Oesterheld)

- **ε -grounded FairBot:** with small probability ε , cooperate *unconditionally* (“ground out”); with probability $1 - \varepsilon$, *run the opponent’s program* on your own code and *copy* its action.
- **Why it terminates:** each level of mutual simulation grounds out with probability ε , so the recursion halts with probability 1 – no infinite regress (contrast the naive “run each other” that hangs).
- Two ε -grounded FairBots (almost always) cooperate, and this is a **Nash equilibrium** of the program game. It is **robust**: programs that *behave* the same are treated the same (unlike syntactic proof-based bots).
- (The ε -grounded π Bot and its generalisations, arXiv:2412.14570, prove folk theorems for such simulation-based equilibria.)



What open-source game theory is really doing

Conditional commitment

- Two desiderata for a good decision theory here:
 - (1) **cooperate with your own copy**, and
 - (2) **unexploitability** – an opponent cannot become better off by making you worse off.
- Why ordinary game theory fails: payoff depends on the joint action, but you are uncertain about the opponent's action and they about yours – a chicken-and-egg of *mutual uncertainty*.
- **Open-source game = conditional commitment**: “I commit to X conditional on you committing to Y.”
 - the **conditional** part keeps you *unexploitable* (you cooperate only if they do);
 - the **commitment** part makes your cooperation *legible*, so you can satisfy the conditions of the opponent's conditional commitment.
- FairBot is exactly this: “I commit to C conditional on a proof that you commit to C.”



The commitment races problem

Why consequentialists race to commit (Kokotajlo)

- A consequentialist (acts purely on expected consequences) is both a **bully and a coward**: it threatens when threats work, and *caves* to credible threats. So **whoever commits first controls the one who commits later**.
- This creates a **race**: it pays to commit *before* you have thought or learned – to “move first”. For agents that read or simulate each other, even *thinking longer* leaks exploitable information about what you will do.
- Two catastrophe modes:
 - **Incompatible commitments lock in**: both players throw out the steering wheel in a game of Chicken $\Rightarrow$ a crash neither wanted (e.g. grim-trigger protocols keyed to incompatible bargaining solutions).
 - **Wasteful racing**: everyone commits before learning – a coordination failure like an arms race.



Moving first in logical time; and s-risks

Why this is a safety priority

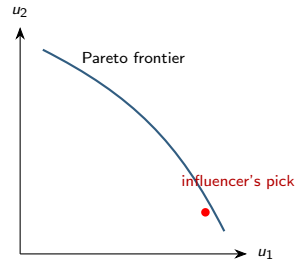
- “First” is not clock time but **logical time**: who, in the structure of mutual prediction, effectively decides *before* the other. An agent that is *predicted* to have committed *is* committed – even simultaneously in clock time.
- **Connection to s-risks (risks of astronomical suffering)**. The most dangerous commitments are *threats*: “do X or I make things very bad for you.” Commitment races can lock superintelligences into *carrying out* catastrophic threats (or executing them to build credibility), i.e. suffering on an astronomical scale.
- So a decision theory that avoids commitment races – letting agents reach good outcomes *without* a destructive scramble to pre-commit – is a genuine safety goal, not a curiosity.



Influencer vs. responder

The first-mover's advantage, and the race it triggers

- **Responder:** run the opponent's program, observe its action, then $\arg \max$ your own utility against that action (a plain *best response*).
- **Influencer:** assume the opponent is a responder (will best-respond to you); $\arg \max$ your utility over that – i.e. pick the point on the Pareto frontier **best for you and worst for the opponent**, knowing they will cave.
- **"The best response is not the best response":** being a naive best-responder is *dominated* by being the influencer. So everyone wants to be the influencer $\Rightarrow$ a race to "move first in logical time".



moves first $\Rightarrow$ grabs its favourite corner of the frontier



The generalized commitment race: entanglement

Why agents are not incentivised to make Pareto improvements

- When you choose your program in an open-source game, you optimise against your *subjective distribution* over the opponent's possible programs – including **counterfactual** ones (programs they might have submitted but didn't).
- So you cannot simply maximise against the opponent's *actual* program: doing so might make you worse off against *counterfactual* versions. Your incentive is **entangled** with those counterfactual programs.
- Equivalently: whenever a rational agent “*punishes*” an opponent (for, say, proposing a Pareto improvement), it is *forgoing utility against the actual program* to gain utility against some *counterfactual* program.
- **Entanglement is the obstacle:** it is exactly what disincentivises accepting a Pareto improvement (accepting might look “soft” and cost you against a counterfactual hawk). This is the *generalized* commitment race – it persists even with full conditional commitment.



Safe Pareto improvements (SPI)

Improving on a default that risks conflict

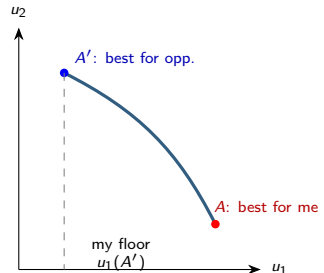
- Setup (Oesterheld–Conitzer; DiGiovanni–Clifton–Macé, arXiv:2403.05103): players delegate to *representative* programs. The *default* instruction is “play the original game as best you can”.
- An **SPI** is a re-instruction (restrict the strategy set, or re-map utilities) that is **guaranteed to make every player weakly better off** – with *no* probabilistic assumptions about how the representatives play. A Pareto improvement that is *safe* because it improves *regardless* of the opponent.
- Example: a game heading for costly conflict (Chicken-style). Agree on a transformation that strips out the mutually destructive outcome; each player does at least as well, whatever happens.
- The paper proves a clean guarantee – the **Pareto Meet Minimum (PMM)**: each player gets at least their *lowest payoff over efficient (Pareto-optimal) outcomes*, and this bound is *tight* (renegotiation cannot promise more). Next slide: why. Caveat: it needs a strong assumption.



Why an SPI guarantees a floor: the Pareto Meet Minimum

The shape of the renegotiation set controls exploitation

- Each agent's *renegotiation set* is the Pareto-improvements it will accept. On the frontier, "better for the opponent" means "worse for me".
- Let A be the frontier point **best for me** (worst for the opponent). I have every reason to include points *up to* A : pushing the opponent below A 's level cannot help me – by definition A already maximises *my* payoff on the frontier.
- But including points *beyond* A (where I would accept being worse than at A) enables **exploitation**: the opponent could make herself better off by making me worse off.
- So rational renegotiation sets stop at each player's A -point. The worst the agreed outcome can be for me is the *opponent's* A -point – my **lowest efficient payoff**. That guaranteed floor is the **PMM** (and it is tight).



accept the frontier between A' and A , never
beyond



The assumption SPI relies on – and the cheating problem

Participation independence

- **Participation independence:** representatives behave the *same* in the SPI-transformed game as in the default game – their play does not change just because an SPI is in effect. This independence is what lets you *guarantee* the improvement.
- But it is unrealistic. **The cheating problem:** if I know that an SPI will Pareto-improve away any conflict, then conflict is *less costly* for me, so I am incentivised to submit a *more hawkish* default program.
- Anticipating my hawkishness, my opponent's incentive to offer the SPI in the first place is undermined. **The SPI can defeat itself** – precisely the entanglement effect, reappearing.



Entanglement-free SPI: the idea

Make the improvement with an entanglement-free program

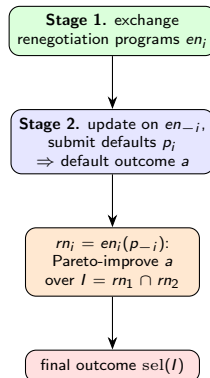
- Diagnosis: *entanglement* (caring about counterfactual opponent programs) is what disincentivises Pareto improvements. So make the improvement using a separate program that is **entanglement-free** – the **renegotiation program**.
- **Sequential structure:** agents *first* exchange renegotiation programs and *update on the actual one*; *then* submit their default programs.
- Because you update on the opponent's *actual* renegotiation program, you optimise against *it*, not against counterfactual versions $\Rightarrow$ *no entanglement* $\Rightarrow$ no incentive to punish/reject improvements.
- The renegotiation program may *only propose Pareto improvements* (conditional on the opponent's default program). Two consequences:
 - it can *never make the opponent worse off* $\Rightarrow$ no influencer-responder exploitation, *even though* it moves first;
 - it *conditions on the default program* $\Rightarrow$ it can *detect* hawkish “cheating” defaults and withhold improvements, restoring unexploitability and **removing reliance on participation independence**.
- You keep your full action space: you can still play the open-source game with your default program; the renegotiation program only *adds* the option to Pareto-improve.



Entanglement-free SPI: the protocol

A clean formalism

- Open-source game: actions $A = A_1 \times A_2$, utilities $u_i : A \rightarrow \mathbb{R}$, programs $p_i : P_{-i} \rightarrow A_i$, joint action $a = (p_1(p_2), p_2(p_1))$.
- **Renegotiation function** $rn_i : RN_{-i} \times A \rightarrow \mathcal{C}(A)$: conditional on the opponent's renegotiation function, it proposes a set of Pareto-improvements on the default outcome a . A rule sel picks one point of the agreed intersection.
- **Entanglement-free renegotiation program** $en_i : P_{-i} \rightarrow RN_i$ reads the opponent's default program, outputs a renegotiation function.
- The intersection $I = rn_1(rn_2, a) \cap rn_2(rn_1, a)$ holds the Pareto-improvements *both* agents will make; $\text{sel}(I)$ is the final outcome. (Choosing sel adds no coordination problem, under mild conditions – DiGiovanni.)





Why first-mover is safe here – and what we trade away

Resolving the race without creating a new one

- Usually, a sequential structure creates an influencer-responder race (the first mover grabs the frontier). Here the first move (renegotiation) can *only Pareto-improve*, so it **cannot exploit** – no new commitment race is introduced.
- Net result: entanglement-free SPI **solves the cheating problem** and **drops the participation-independence assumption** (the renegotiation program conditions on the defaults to catch cheating), while the sequential structure **avoids introducing its own commitment race**.
- **The trade-off – no tight PMM.** The renegotiation program conditions on the default program, so it can create an *incentive gradient*: “more cooperative default $\Rightarrow$ I offer more Pareto-improvement; more hawkish $\Rightarrow$ less”. Because it may *withhold* improvement to shape the opponent’s default, the agreed set is no longer the full “up-to-A” set, and the tight **Pareto Meet Minimum** floor is lost. Weaker (more realistic) assumptions buy a weaker guarantee.

(Entanglement-free SPI: Daniel C; selection-function coordination after DiGiovanni.)